

Sprint 1 SPEC: 全路径自适应 FNOL Prototype

1. 文档状态

- **Sprint:** Sprint 1
- **日期:** 2026-08-10 至 2026-08-14
- **展示:** 2026-08-14 Prototype Presentation
- **团队容量:** 5 人, 每人 40 小时, 总计 200 人时
- **产品依据:** [project_soul.md](#)
- **状态:** 待团队确认后拆分 Backlog 与成员任务

2. Sprint Goal

中文: 在 Sprint 1 结束前, 构建并演示一个路径覆盖完整、实现深度受控的自适应 FNOL Prototype。它应从客户自然语言报案开始, 维护可确认的结构化 claim form, 使用 AWS 提供的数据处理 policy、材料和历史 claim, 并在快速、复杂、紧急、人工请求、待补材料和跨会话场景中选择安全的下一步, 最终创建或正确转交 claim; 系统同时维护内部 claim state、tag 和 Claim Operations Workbench, 并记录 token 与人工 effort。

English: By the end of Sprint 1, build and demonstrate a breadth-complete, depth-limited adaptive FNOL prototype that turns a claimant's natural-language report into a confirmable structured claim form, uses AWS-provided policy, evidence and claim-history data, selects a safe next action across fast, complex, urgent, human-requested, pending-evidence and resumed-session paths, and either creates or correctly hands off the claim while maintaining internal claim state, tags and a Claim Operations Workbench and recording model and human effort.

3. Prototype 定义

Prototype 必须展示未来产品的核心路径骨架, 不能只完成一个 happy path。允许受控规则、有限数据、mock services 和简化界面, 但每条核心路径必须真实改变系统状态并产生可验证结果, 不能只用静态幻灯片或口头说明代替。

4. Actors 与系统边界

- **Claimant:** 描述事件、回答必要问题、上传材料、确认或纠正 form、查看状态、继续会话或请求人工。
- **FNOL Agent:** 理解、提取、澄清、查询、决定下一动作、更新状态、调用工具和组织 handoff。
- **Claims Professional / Fraud Professional:** 接收需要人工判断的结构化上下文并执行指定动作。
- **Claims Operations user:** 通过系统维护的 Workbench 查看队列、筛选 claim、处理 tag/signal、分配工作并推进下一动作。
- **AWS-backed data and tools:** policy documents、历史 claim、session/claim persistence、mock claims system 和 mock assessor booking。

5. Functional Requirements

FR-01 身份、Claim 与 Session

- 系统必须创建或识别 `customer_id`、`claim_id` 和 `session_id`。
- 同一 customer 可以有多个 claim，同一 claim 可以跨多个 session。
- 用户恢复会话时必须回到当前 claim state 和未解决问题，不重新开始。

FR-02 对话式 Form

- Claimant 可以用自然语言描述事件。
- Agent 必须把信息写入结构化 form，而不是只保留聊天文本。
- Form 字段必须记录值、来源、状态、用途和更新时间。
- 用户必须能在请求时以及关键确认、创建和转交节点查看、确认和纠正关键字段。

FR-03 自适应提问

- Agent 只询问当前 next safe action 需要的信息。
- 已确认字段不得无原因重复询问。
- 对歧义、冲突、缺漏或低质量材料执行针对性澄清。
- Agent 必须允许“不知道”“稍后提供”和无法继续回答，不可猜测或给出违反policy的回复。

FR-04 多模态材料

- Prototype 必须接收至少图片和 PDF。
- 图片或文件提取结果必须关联到 evidence 和相关 form 字段。
- 提取内容必须标记为材料来源，并允许用户确认或纠正。

FR-05 Policy 查询

- Agent 必须查询 AWS 提供的结构化 policy 数据和 anonymised policy documents。
- Coverage/excess 输出必须带检索依据或明确表示不确定。
- 无法安全判断时必须转交 Claims Professional，不得伪造确定结论。

FR-06 快速与复杂路径

- 明确信息和规则满足条件时，Agent 应减少问题并进入快速路径。
- 复杂、冲突或高影响决定应进入引导或专业判断路径。
- 路径选择必须留下 reason code 和可审计记录。

FR-07 紧急路径

- 明确人员伤亡、持续危险或紧急援助需求必须中断普通 FNOL。
- 系统必须提供清晰安全提示并生成高优先级人工 handoff。
- Prototype 不进行医疗诊断，也不假装已联系紧急服务。

FR-08 用户请求人工

- 系统必须识别明确的人工请求及可能原因。
- 可提供一次简短、透明的继续自助选择，但不得形成阻止转交的循环。
- 重复请求、无障碍需要、明显 distress 或紧急情况必须立即转交。
- 具体首次请求策略在实现前由团队确认。

FR-09 Next-action-ready 与材料状态

- Evidence 至少支持 `received`、`incomplete`、`missing`、`pending_generation` 和 `needed_later`。
- 缺少未来材料不得自动阻塞当前不依赖该材料的安全动作。
- Agent 必须说明材料何时需要、由谁提供以及如何在同一 claim 上补充。

FR-10 持久记忆与 Token 控制

- 完整消息保存在外部持久层，不在每轮全部放入模型上下文。
- 每轮上下文使用当前 claim snapshot、未解决问题、最近必要消息和相关检索结果。
- 系统必须维护结构化摘要，并演示模拟相隔 8-10 天后恢复会话。
- 每轮记录 input/output tokens、检索量、摘要或压缩事件。

FR-11 历史 Claim 与 Fraud Signal

- Agent 必须查询当前客户的相关历史 claim 数据。
- Signal 必须关联支持信息和 reason code。
- Signal 只进入 Fraud Professional review，不自动认定欺诈或向客户作出指控。
- Fraud review 不应无依据阻塞 claim 创建。

FR-12 Claim 创建、路由与 Assessor

- 达到创建条件时，Agent 必须调用 mock claims system 创建 claim。
- 返回并展示 claim number、route、状态和 expected timeline。
- Assessor booking 仅在受控规则判定需要时调用 mock tool。

FR-13 Context-preserving Handoff

- Handoff packet 必须包含已确认摘要、form、evidence、来源、缺失/冲突信息、转交原因、优先级和请求人工执行的动作。
- Staff view 必须能消费该 packet。
- 测试必须证明人工无需重新询问 Prototype 已经确认的关键信息。

FR-14 状态与客户更新

- 每次暂停、转交或结束必须说明当前状态、客户是否需要行动、下一责任方和预计时间。
- 待补材料和未来动作必须在 claim 状态中可见。

FR-15 多维 Claim State 与内部 Tag

- 系统必须分别维护 `severity`、`coverage`、`evidence`、`fraud_signal`、`customer_support`、`urgency`、`workflow_state` 和 `next_action`，不得用单一互斥路径覆盖全部状态。
- 同一 claim 必须支持多个可组合属性，例如 `severity: standard`、`coverage: clear`、`evidence: police_report_pending` 和 `next_action: create_claim`。
- 内部 tag/signal 必须记录 code、category、visibility、source、evidence refs、confidence、status、required action、assigned queue 和审计时间。
- 高影响 signal 初始状态只能是 `proposed` 或 `review_required`；员工必须能够 confirm、dismiss、override 或 resolve。
- 内部敏感 signal 不得直接暴露给 claimant；客户只接收适当的状态和下一步说明。

FR-16 Claim Operations Workbench

- 系统必须根据 claim state 自动维护 Workbench，不要求员工重复录入第二套看板数据。

- Workbench 至少提供 Urgent、New/Untriaged、Ready to Progress、Awaiting Evidence、Professional Review、Ready to Create 和 Created/Routed 视图。
- 企业端用户必须能按状态、tag、priority、assignee、next action 和 SLA 筛选。
- Claim 详情必须显示 form、evidence、来源、冲突、内部属性、handoff packet 和客户沟通。
- 员工必须能分配 claim、处理 proposed tag/signal、执行下一动作并将结果写回 claim state。
- Workbench 处理结果必须触发适当的 claimant 状态更新。

6. Agent 决策契约

每轮输出至少包含：

```
action
reason_code
claim_state_changes
internal_attributes
internal_tags_proposed
visibility
questions_or_message
required_tools
next_action_requirements
handoff_priority
customer_visible_next_step
```

允许动作：ASK、CLARIFY、CONFIRM、PROCEED、UPDATE、HANDOFF、URGENT_HANDOFF、CREATE_CLAIM。

高影响动作的授权与规则检查必须在模型输出之外执行；模型建议不能直接绕过 policy 或专业判断边界。

7. 最小数据契约

Prototype 至少需要：

- `customers`：用户标识和允许保留的偏好；
- `claims`：claim form、workflow state、route 和 current next action；
- `claim_attributes`：severity、coverage、evidence、fraud/customer support、urgency 和 next action 等可组合状态；
- `internal_tags`：来源、证据、可见性、状态、处理队列和审计记录；
- `sessions`：会话状态和结构化摘要；
- `messages`：完整对话记录；
- `evidence`：材料、来源、状态和字段关联；
- `decisions`：动作、reason code、依据和执行结果；
- `handoffs`：人工交接 packet 与状态；
- `staff_actions`：assignment、confirm/dismiss/override、处理结果和 claimant update；
- `claim_events`：可审计状态变化。

不为每个用户创建独立物理表；通过 `customer_id`、`claim_id` 和 `session_id` 分区和关联。

8. Acceptance Scenarios

ID	场景	必须观察到的结果
AT-01	明确的轻微 motor 事故	少量针对性问题、form 确认、快速推进并创建 mock claim
AT-02	Policy wording 或事件适用性模糊	展示依据和不确定性，携带上下文转交 Claims Professional
AT-03	复杂事件或证据冲突	记录冲突、避免武断结论、请求专业判断
AT-04	明确人员受伤或现场危险	普通流程被中断，给出安全提示并生成 urgent handoff
AT-05	Claimant 请求人工	按确认后的策略响应，最终转交且不丢失上下文
AT-06	警方文件尚未生成	标记 <code>pending_generation</code> ，推进不依赖该文件的动作并说明补充方式
AT-07	图片包含可提取事故信息	提取为 proposed 字段，用户可确认或纠正
AT-08	模拟十天后继续	恢复 claim snapshot、待办和上一承诺，不重复开始
AT-09	历史 claim 产生 supported fraud signal	Signal 附依据进入专业复核，不作欺诈结论
AT-10	需要 assessor 的受控场景	创建并路由 claim，调用 mock assessor 并显示 timeline
AT-11	多个 claim state 属性同时存在	待生成材料不会覆盖 clear coverage 或错误阻塞 <code>create_claim</code> ，属性和 next action 均可追溯
AT-12	内部 signal 进入 Workbench	Claim 自动进入正确队列；员工查看依据、确认或覆盖 signal、完成动作，结果写回并更新 claimant

这些是 Sprint 1 的路径验收集，不替代最终 Brief 要求的 5 motor、3 home、2 contents 测试集。

9. Non-functional Requirements

NFR-01 可追溯性

关键字段、policy 判断、fraud signal、内部 tag、员工 override、路由和 handoff 必须能追溯来源与 reason code。

NFR-02 成本可观察

Prototype 必须记录 token、工具调用、人工介入原因、事件/执行开销、认知开销和 handoff packet 完整度，为后续优化建立基线。

NFR-03 演示可靠性

全部 acceptance scenarios 必须使用可重复 fixture。核心演示路径不能依赖临时手工改数据。

NFR-04 数据边界

不得把 secret、真实个人信息或非必要完整历史记录写入 prompt、日志、文档或展示材料。

NFR-05 用户控制

客户必须能纠正 form、知道 Agent 的不确定性、了解下一步并获得人工路径。

NFR-06 权限与可见性

Prototype 必须区分 customer-visible、shared 和 internal-only 信息；敏感 tag/signal 只对授权企业端用户可见。

10. Effort Metrics

Prototype 至少输出：

- 每场景问题数、完成时间、纠正次数和重复询问次数；
- 每场景 token、检索和工具调用量；
- handoff rate、reason、priority 和 packet 完整度；
- 人工介入次数、处理时长、转派和 follow-up 次数；
- 模拟人工接手后需要重新询问的字段数量，以及理解 initial claim 和未决问题所需时间；
- 从恢复 session 到执行下一有效动作所需步骤。

11. Delivery Artifacts

- 可运行的 claimant-facing Prototype；
- 可见且可纠正的 structured form；
- Agent orchestration 与动作/状态契约；
- AWS 数据接入、policy RAG、session/claim persistence 与 mock tools；
- 系统维护的 Claim Operations Workbench，包括队列、内部属性/tag、handoff 处理和 claimant update；
- acceptance fixtures、测试结果和 effort 指标；
- Friday 演示脚本和可重复演示环境；
- Sprint 1 多泳道汇流图：中文 [draw.io 源文件](#) / [slide-ready SVG](#)，英文 [draw.io source](#) / [slide-ready SVG](#)。

12. workflows 框架（暂不分配成员）

以下五个工作流是五类持续推进的工作，不是五个固定岗位。成员可以根据当前阻塞和任务进度在工作流之间移动；每个具体任务仍必须只有一个当期 owner。

工作流 A：产品规则与 Agent 行为

这条工作流先回答“系统在什么情况下应该做什么”。主要工作包括：

- 把快速、复杂、紧急、人工请求、材料待补和跨会话场景写成明确输入与预期结果；
- 定义 form 字段、claim state、内部 tag、next-action-ready 和 handoff 条件；
- 定义 Agent 可执行动作、禁止行为、reason code 和客户可见文案边界；
- 确定每个 acceptance scenario 怎样才算通过。

主要产物是行为规则、场景 fixture、验收条件和共享契约。前端、Agent、数据和测试工作都依赖这些产物，避免各自猜测业务逻辑。

workflow B: Claimant 与 Staff 使用体验

这条 workflow 负责用户实际看到和操作的界面：

- Claimant 端的对话、form 查看/纠正、图片/PDF 上传、材料状态、进度和下一步；
- Staff 端的 Claim Operations Workbench、队列、筛选、claim 详情、tag/signal 处理和 handoff；
- 紧急提示、人工请求、错误状态、等待状态和跨会话恢复体验；
- 保证同一 claim state 在 claimant 与 staff 两端显示为适合各自权限和任务的信息。

主要产物是可操作界面和交互状态。它使用 workflow A 的行为规则以及 workflow D 的数据/API，不自行发明业务判断。

workflow C: Agent 运行与流程编排

这条 workflow 负责让 Agent 按规则工作，而不是只生成一段回复：

- 根据当前 claim state 选择 ASK、CLARIFY、PROCEED、HANDOFF 等动作；
- 调用 policy、历史 claim、evidence、claim creation 和 assessor 工具；
- 维护 session、结构化摘要和每轮模型上下文；
- 控制 token、重试、错误处理和高影响动作的规则校验；
- 把执行结果写回统一 claim state，并提供给前端和 Workbench。

主要产物是 Agent action contract、编排逻辑、工具调用和状态转换。

workflow D: 数据、知识与 AWS 集成

这条 workflow 负责“数据从哪里来、保存在哪里、怎样被其他模块调用”：

- 检查 AWS 提供的数据和 mock services，建立适配层；
- 保存 customer、claim、session、messages、evidence、tags、handoffs 和 events；
- 建立 policy 查询/RAG、历史 claim 查询和 fraud signal 所需的数据接口；
- 接入图片/PDF 存储、mock claim creation 和 assessor booking；
- 提供部署、权限、日志和环境配置。

主要产物是数据 schema、API/tool contract、持久化和 AWS 云端运行环境。

workflow E: 交叉测试、集成与交付

这条 workflow 不是把测试全部交给一个人，而是协调所有模块持续合并：

- 把 A-D 的产物接成端到端流程，每天验证当前可运行版本；
- 用固定 fixture 运行 AT-01 至 AT-12，记录实际结果和缺陷；
- 检查 claimant、Agent、数据和 staff Workbench 对同一 claim state 的理解是否一致；
- 收集 token、人工 effort、错误、延迟和 handoff 完整度；
- 维护云端演示环境、回归清单、演示脚本和备用演示路径。

主要产物是持续可运行的集成版本、测试证据和最终 Prototype。测试责任仍属于每个任务 owner，workflow E 负责跨模块验证和交付协调。

当前多泳道汇流基线已使用 draw.io 绘制并维护中英文对照版本，横轴为 Day 1 至 Day 5，纵轴为产品契约、Claimant 体验、Staff Workbench、Agent 编排、数据/AWS、集成测试与交付六条能力流；泳道不绑定固定成员。源文件与 SVG 见 [Delivery Artifacts](#)。具体成员姓名与 1-2 小时叶子任务继续在 Kanban 动态分配，所有能力流最终汇合到 **Prototype Delivery**。

13. 协作与集成门

集成门是全队检查“当前产物能否连接并运行”的时间点，不是等待某位负责人批准的行政流程。

Gate 1: 共同语言和接口可用（周一）

周一先确定所有人共同使用的 claim state、form/data schema、Agent action contract、API/tool contract 和 acceptance fixtures。完成后，前端、Agent、数据和测试可以使用同一套字段、动作和场景并行工作。

Gate 2: 最小端到端骨架可运行（周二）

Claimant 输入一段自然语言后，系统能够更新 form、持久化 claim、调用一个 mock tool，并在 claimant 界面和 Workbench 看见同一状态。此时功能可以很简单，但数据必须真实流过所有主要层。

Gate 3: 所有核心分支接通（周三）

快速、复杂、紧急、人工请求、材料待补和 session 恢复都能进入正确 next action；claim state 和内部 tag 能自动进入 Workbench，staff 的处理结果可以写回。

Gate 4: Prototype 功能冻结（周四中午）

AT-01 至 AT-12、effort 指标、内部 tag 处理、staff handoff 和 claimant update 形成闭环。从这一时间点起停止增加功能，只修复阻断演示或破坏验收的缺陷。

周四下午完成回归和演示排练；周五只进行最终检查和 Prototype Presentation。

依赖如何管理

“依赖”指某任务开始或完成前必须先获得的输入。例如：

- 前端和后端共享同一 claim state 前，必须先确定字段和状态名称；
- Workbench 开发依赖 claim attributes、internal tags 和 staff actions 契约；
- Policy/history 路径依赖 AWS 数据适配器或可替代的 fixture；
- Agent 路由测试依赖行为规则、reason code 和预期结果；
- 最终演示依赖所有核心路径进入同一个云端集成版本。

只有依赖已经满足的任务才进入 **Ready**。遇到新依赖时，在任务中写清“等待什么、由哪项任务提供、影响哪个验收场景”，而不是让成员自行等待。

Backlog 使用四级结构：Sprint Goal → capability outcome → verifiable work package → 1-2 小时叶子任务。每项叶子任务有一个当期 owner；reviewer 根据该任务影响的接口、当前可用人员和需要的交叉视角动态选择，不设置固定 reviewer 配对。成员同时最多一个 **In progress** 任务。

14. Out of Scope for Sprint 1 (sprint1不考虑的)

- 生产级安全、容量、灾备和全量合规实现；
- 覆盖所有真实 policy wording 和异常数据；
- 最终十场景的完整准确率承诺；
- 生产 claims/assessor system 上的真实写操作；
- 自动欺诈认定、claim 批准/拒绝或医疗诊断；
- Claim 创建后的完整生命周期追踪；
- 未经确认的语音能力；
- 最终技术架构、固定人员职责和最终 slide 模板适配。

15. Definition of Done

Sprint 1 只有在以下条件全部满足时完成：

- AT-01 至 AT-12 均能重复执行并有结果记录；
- 核心路径不是静态演示，系统状态和工具调用随输入改变；
- Claimant 能看到并纠正 form；
- Policy、history、evidence 和 session 数据均通过定义的接口参与至少一个路径；
- 快速、专业判断、紧急、人工请求、待补材料和恢复路径均可端到端演示；
- Mock claim 创建、route、claim number、next step 和 timeline 可见；
- Staff handoff packet 可被 staff view 消费，且不要求重新采集已确认信息；
- Workbench 由 claim state 自动维护，员工可以筛选、处理 proposed tag/signal、执行下一动作并写回结果；
- 内部敏感 signal 与 claimant-visible status 正确隔离；
- Token 和人工 effort 指标可查看；
- 无 secret、真实个人信息或无依据的 coverage/fraud 结论；
- 演示环境在功能冻结后完成一次完整回归和排练。

16. 已知约束、技术建议与待确认事项

已知约束

- AWS 数据和 mock services 的实际 schema、访问方式与可用性暂不可知；接入前使用与预期契约一致的 fixture 和 adapter，拿到真实数据后替换 adapter。
- 技术栈由团队决定；部署目标是 AWS 提供的云端环境。
- 正式代码仓库尚未建立。
- 五名成员不绑定固定能力岗位，按 workflow 进度灵活领取任务；reviewer 按任务动态选择。
- 多泳道汇流图已形成 16:9 工作基线，最终节点为 **Prototype Delivery**；依赖关系来自实际任务输入/输出，后续可按最终 slide 模板等比适配。

推荐技术基线（待团队确认）

- **前端**：React + TypeScript + Vite，部署到 AWS Amplify Hosting；
- **Agent/API**：Python + FastAPI，以显式 claim state 和 action contract 控制流程；标准 API 使用 API Gateway + Lambda，Agent 长任务或流式运行优先评估 Bedrock AgentCore Runtime；
- **模型**：Amazon Bedrock，具体模型在数据和账号权限明确后选择；
- **数据**：DynamoDB 保存 claim/session/state/event，S3 保存图片 and PDF；
- **Policy 查询**：根据 AWS 数据形式选择 Bedrock Knowledge Bases/OpenSearch 或直接结构化查询；
- **身份与权限**：Amazon Cognito；
- **日志与指标**：CloudWatch，记录 token、工具调用、路径、人工事件/执行和认知开销；
- **基础设施**：AWS CDK，保证云端环境可重复部署。

该建议优先保证一周内能形成云端全路径 Prototype；取得 AWS 实际数据、权限和服务限制后再最终确认。

仍需确认的业务规则

- **首次人工请求策略**：当 claimant 第一次说“我想找人工”时，是立即转交，还是允许 Agent 用一句话说明“可以现在转交，也可以先完成当前这一步”并由用户选择。重复请求、紧急、distress 或无障碍需要仍应立即转交。
- **受控业务规则**：Prototype 为了稳定演示而明确写死或配置的触发条件，例如“明确有人受伤 → urgent handoff”“coverage 无法从给定依据确认 → professional review”。它们是测试规则，不伪装成 Northwind 最终生产规则。

- **Fixture:** 为某条路径准备的匿名或模拟输入数据包, 包括 customer、policy、claim history、对话、图片/PDF 状态和预期结果。Fixture 让团队每次都能重复触发同一路径并判断是否通过。
- **具体规则值:** coverage、severity、fraud signal、assessor booking 和首次人工请求的最终触发条件, 需结合 AWS 数据和导师反馈确定。
- **Draw.io 版式:** 当前采用 16:9、Day 1-5 横轴和六条能力流泳道; 最终 slide 模板确定后只做尺寸与密度适配, 不改变 Gate 与依赖语义。